SIMATS ENGINEERING

ASSESSMENT TOOL 1 – Comparative Analysis

COURSE NAME: Data Structures COURSE CODE:CSA0316

COURSE OUTCOME COVERED

CO4: Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data **(BL3)**

Assessment Tool Weightage: 30%

QUESTIONS: EACH QUESTION CARRIES: 10 MARKS DURATION : 1 HOUR

17.Compare binary heaps and Fibonacci heaps for implementing Dijkstra's shortest path algorithm on a dense graph with 100,000 nodes and 10 million edges. Analyze amortized time complexity for decrease-key operations, memory overhead, and practical performance. Why is Fibonacci heap rarely used in practice despite better theoretical bounds?

18.Compare Timsort (adaptive hybrid sort) versus traditional merge sort for sorting real-world data with the following characteristics: (a) already sorted array, (b) reverse sorted array, (c) array with many duplicate keys, (d) array with random order. Analyze best-case, average-case, and worst-case time complexity for each scenario.

19.Compare hash table with separate chaining versus binary search tree for implementing a dictionary in a multi-threaded environment. Analyze locking granularity, concurrent access patterns, resizing complexity, and cache behavior. Which structure supports more scalable concurrent operations?

Code of Conduct:

I G.Hima Bindu(192511377)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G.Hima Bindu

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

17.Compare binary heaps and Fibonacci heaps for implementing Dijkstra's shortest path algorithm on a dense graph with 100,000 nodes and 10 million edges. Analyze amortized time complexity for decrease-key operations, memory overhead, and practical performance. Why is Fibonacci heap rarely used in practice despite better theoretical bounds?

Binary Heap vs. Fibonacci Heap for Dijkstra's Algorithm

For a graph with 100,000 vertices (V) and 10 million edges (E), the graph is relatively dense. Dijkstra's algorithm repeatedly performs extract-min and decrease-key operations.

Feature	**Binary Heap**	**Fibonacci Heap**
Insert	O(log V)	O(1) amortized
Extract-Min	O(log V)	O(log V) amortized
Decrease-Key	O(log V)	O(1) amortized
Dijkstra total	O((V+E) log V)	O(E + V log V)
Memory overhead	Low	High
Implementation	Simple	Complex
Cache locality	Better	Poorer
Practical performance	Usually faster	Often slower

Time complexity

Binary heap:

Dijkstra performs approximately E decrease-key operations and V extract-min operations:

For the given graph:

So approximately:

Fibonacci heap:

Decrease-key takes **O(1) amortized**, giving:

Thus, theoretically, Fibonacci heaps are much better for graphs with many edges.

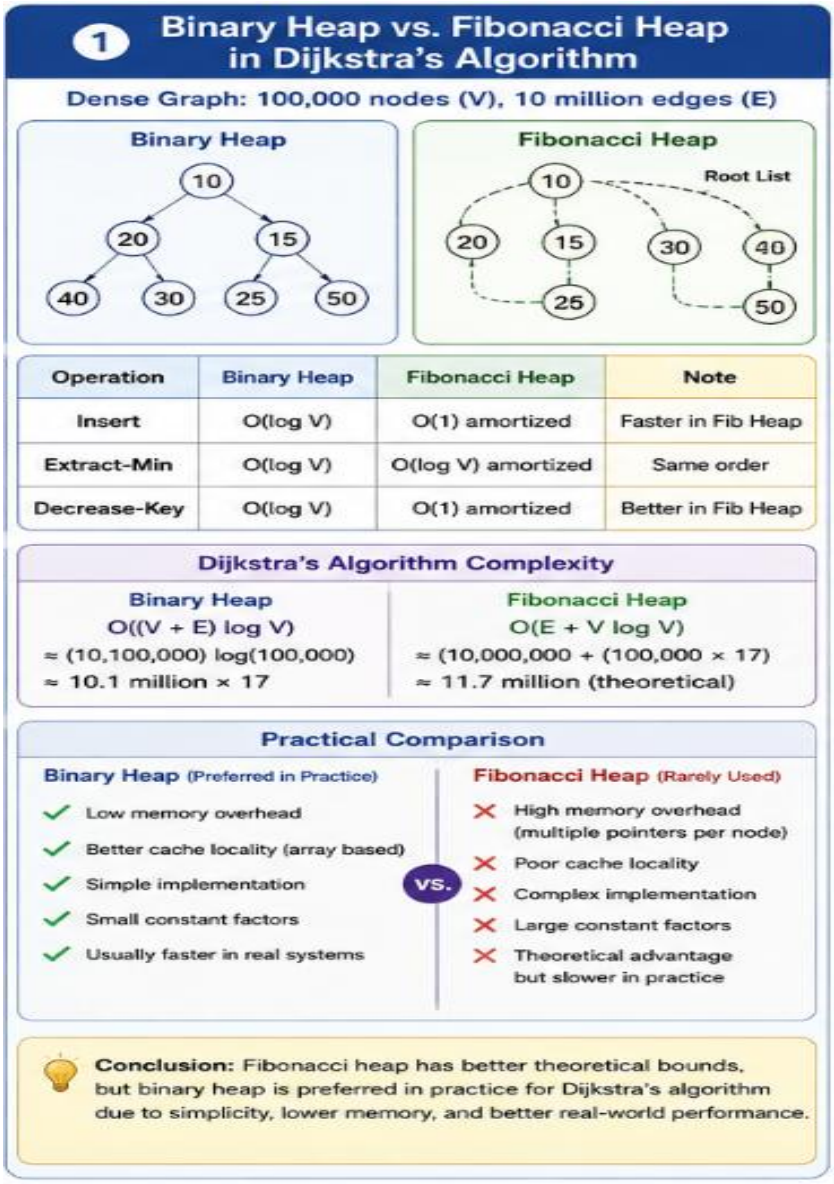
Why Fibonacci heaps are rarely used in practice

Although Fibonacci heaps have a better theoretical bound, they have several practical disadvantages:

1. **High memory overhead:** Fibonacci heaps use multiple pointers per node.
2. **Poor cache locality:** Nodes are dynamically connected and may be scattered throughout memory.
3. **Complex implementation:** Maintaining cascading cuts and heap structure is complicated.
4. **Large constant factors:** The theoretical O(1) decrease-key has overhead.
5. **Binary heaps are cache-friendly:** Arrays provide excellent spatial locality.
6. **Real hardware matters:** CPU-cache behavior can outweigh asymptotic improvements.

Conclusion:

For this 100,000-node, 10-million-edge graph, Fibonacci heaps provide a better theoretical complexity, especially because Dijkstra performs many decrease-key operations. However, a binary heap is usually preferred in real applications because it is simpler, has lower memory usage, and provides better cache performance.



18.Compare Timsort (adaptive hybrid sort) versus traditional merge sort for sorting real-world data with the following characteristics: (a) already sorted array, (b) reverse sorted array, (c) array with many duplicate keys, (d) array with random order. Analyze best-case, average-case, and worst-case time complexity for each scenario.

Timsort vs. Traditional Merge Sort

Timsort is an adaptive hybrid sorting algorithm derived from merge sort and insertion sort. It detects already ordered runs in the input and exploits them.

Input	**Timsort**	**Traditional Merge Sort**
Already sorted	**O(n)**	O(n log n)
Reverse sorted	**O(n)**	O(n log n)
Many duplicate keys	O(n) to O(n log n), depending on structure	O(n log n)
Random order	O(n log n)	O(n log n)
Worst case	O(n log n)	O(n log n)
Extra space	O(n) worst case	O(n)

(a) Already sorted array

Example:

1 2 3 4 5 6 7 8

Timsort:

It detects the entire array as a sorted run.

Mergesort:

It still divides the array recursively and performs merging.

Winner: Timsort

(b) Reverse sorted array

Example:

8 7 6 5 4 3 2 1

Timsort detects a descending run and reverses it efficiently.

Traditional merge sort remains:

Winner: Timsort

(c) Array with many duplicate keys

Example:

5 2 5 5 3 2 5 1 5

Timsort can exploit existing ordered runs and efficiently merge equal-key elements. Depending on how the duplicates are distributed, its performance can range from close to **O(n)** to **O(n log n)**.

Traditional merge sort remains:

Winner: Generally Timsort, particularly when duplicates occur in partially ordered run

(d) Random-order array

For random data, Timsort cannot exploit much existing order.

Timsort:

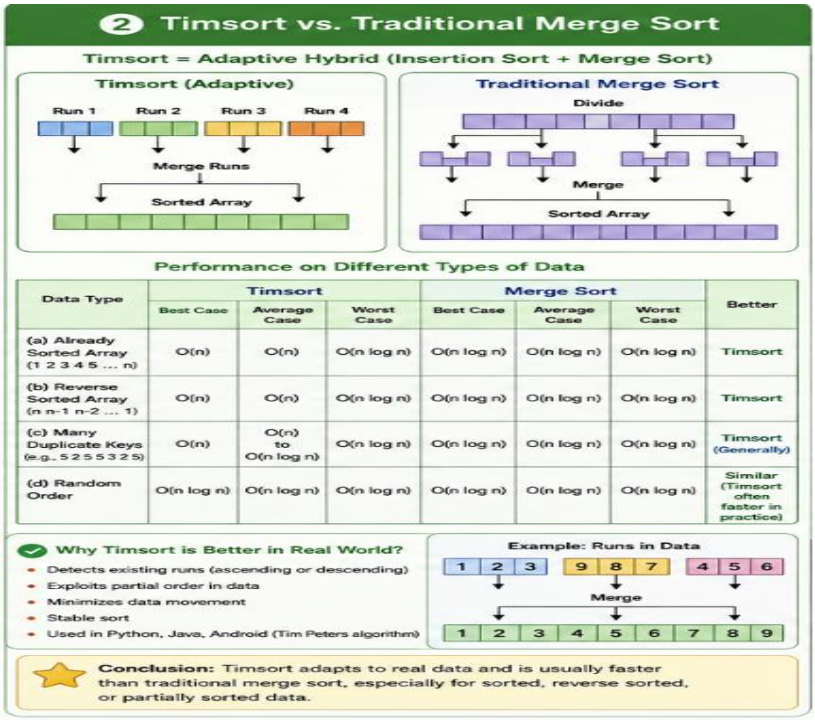
Merge sort:

Both have the same asymptotic complexity, although Timsort may perform better in practice because of its optimized run detection and merging.

Overall comparison

Scenario	Timsort	Merge Sort	Better
Already sorted	**O(n)**	O(n log n)	Timsort
Reverse sorted	**O(n)**	O(n log n)	Timsort
Many duplicates	O(n)–O(n log n)	O(n log n)	Timsort generally
Random	O(n log n)	O(n log n)	Timsort/practical
Worst case	O(n log n)	O(n log n)	Same asymptotically

Conclusion: Timsort is better suited to real-world data, because real data frequently contains partially sorted sequences, repeated values, or existing order. Traditional merge sort provides predictable O(n log n) performance but does not exploit existing order as effectively.



19.Compare hash table with separate chaining versus binary search tree for implementing a dictionary in a multi-threaded environment. Analyze locking granularity, concurrent access patterns, resizing complexity, and cache behavior. Which structure supports more scalable concurrent operations?

Hash Table with Separate Chaining vs. Binary Search Tree

A hash table with separate chaining and a binary search tree (BST) can both be used to implement a dictionary storing key-value pairs.

Feature	**Hash Table**	**Binary Search Tree**
Average search	O(1)	O(log n)
Locking	Bucket-level locking is possible	Node/subtree locking
Concurrent access	High, different buckets can be accessed simultaneously	Lower due to shared tree paths
Resizing	Requires rehashing and synchronization	No resizing required
Cache behavior	Generally better	More pointer chasing and cache misses
Scalability	High	Moderate

Hash table: Separate chaining allows each bucket to have an independent lock. Therefore, multiple threads can perform operations on different buckets at the same time. However, resizing the table requires synchronization and rehashing.

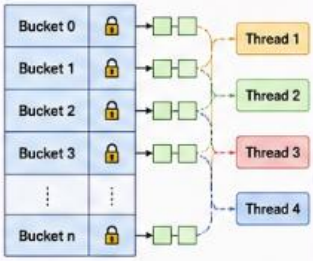
Binary search tree: A BST can use node-level or subtree-level locks, but concurrent operations may still interfere when accessing common parts of the tree. It also has poorer cache locality because tree nodes are usually connected through pointers.

Conclusion

A hash table with separate chaining generally supports more scalable concurrent operations because of its fine-grained bucket locking, O(1) average lookup, and better cache behavior. A BST is preferred when sorted order or range queries are required.

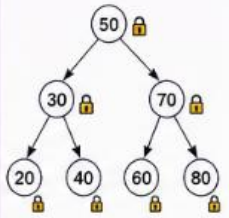
3 Hash Table (Separate Chaining) vs. Binary Search Tree (BST) in Multi-threaded Environment

Hash Table (Separate Chaining)



Different buckets can be locked independently. Multiple threads can access different buckets simultaneously.

Binary Search Tree (BST)



Threads may conflict on common nodes/paths. Requires node/subtree locking.

Key Aspects Comparison

Aspect	Hash Table (Separate Chaining)	Binary Search Tree (BST)
Locking Granularity	Bucket-level locking (Fine-grained)	Node / Subtree locking (Coarse to fine-grained)
Concurrent Access Pattern	High concurrency (different buckets independent)	Lower concurrency (threads interfere on shared paths)
Resizing Complexity	Needs resizing (rehashing). Expensive & requires synchronization.	No resizing required. Node insert/delete only.
Cache Behavior	Better cache locality (buckets in array, lists are local)	Poorer cache locality (pointer-heavy structure)
Average Search / Insert	O(1) average	O(log n) average
Worst Case	O(n) (if many collisions in a bucket)	O(n) (if skewed tree)
Best for	Fast lookups, updates, high concurrency	Ordered data, range queries, sorted traversals
Scalability in Multi-threading	HIGH	MODERATE

🎯 **Conclusion:** Hash table with separate chaining provides more scalable concurrent operations due to fine-grained bucket locking and better cache behavior. BST is preferred when ordered data or range queries are required.